

FamillA is a modern **TanStack Start** web application designed to bridge the gap between financial independence for older adults and peace of mind for their family circles. By providing accessible tools, plain-language translations, and proactive monitoring, FamillA helps seniors manage their finances autonomously while preserving a strong, respectful safety net.

F1: The Core Mission (Foundational Goals)

These objectives establish the primary problem-solving framework.

1.1 Empowering Seniors

SENIORS FOCUS

Providing a highly accessible, simplified companion that listens to their voice, reads back responses, and explains complex documents in plain language.

1.2 Reassuring Families (Tutors)

TUTORS FOCUS

Giving designated family members (tutors) a clear dashboard of activity, wallet monitoring, and fraud-detection indicators without stripping the senior of their agency.

F2: Key Product Features (Implementation Tracks)

Parallel development tracks for core functionality and AI assistance.

2.1 🗣️ Multimodal Copilot

SENIORS

A friendly, accessible chat interface. Seniors can talk, upload images of bills/receipts, or type.

2.2 🔊 Text-to-Speech (TTS)

SENIORS

Auto-reads assistant responses out loud using a natural, localized voice. Supports play, pause, resume, and stop controls to enhance readability and combat visual strain.

2.3 💡 Fraud & Anomaly Alerts

TUTORS

Real-time analysis flags unusual withdrawals, erratic charges, or potential scams immediately.

2.4 📊 Financial Overview

TUTORS

Unified visual dashboard comparing digital cash estimates, bank balances, and upcoming recurring bills.

2.5 📄 Clear Explanations

CROSS-FUNCTIONAL

Translates dense technical bank details, policies, or receipts into friendly, plain-language summaries.

F3: Tech Stack & Architecture Structure

Production-grade React meta-framework architecture.

- **UI Framework:** [React 19](#) with [TypeScript](#)
- **Application Framework:** [TanStack Start](#) (integrating SSR and server-client RPCs)
- **Routing:** [TanStack Router](#) (type-safe, file-based routing)
- **Data Fetching:** [TanStack Query v5](#) (caching and remote states)
- **Styling & Motion:** [Tailwind CSS 4](#), [Framer Motion](#), [Radix UI Primitives](#)
- **Data & Charts:** [Recharts](#) for financial analytics
- **Validation:** [Zod](#) with [React Hook Form](#)

Repository Structure

```
├─ src/
│  ├─ routes/           # File-based router pages
│  ├─ components/       # Shared visual primitives & layout widgets
│  │  └─ ui/            # Low-level UI primitives
│  │     └─ dashboard/  # Dashboard widgets (charts, logs)
│  └─ lib/              # Storage controllers, mock data, global utils
│     └─ router.tsx     # Router configuration & query context
│     └─ start.ts       # Bootstrap entry point for TanStack Start
├─ docs/
│  └─ technical-stack.md # Deep technical breakdown
│  └─ user-guide.md     # Interactive journey walkthrough
```

F4: Integration & Execution Setup

Getting started, CLI workflows, and state persistence.

Getting Started

Prerequisites: Node.js 20.x or Bun 1.x. Modern web browser with Web Speech API support.

1. **Install Dependencies:** Run `bun install` or `npm install`.
2. **Launch Development Server:** Run `bun run dev` and open the printed URL (usually localhost:3000).
3. **Build & Preview Production:** Verify with `bun run build` followed by `bun run preview`.

CLI Reference

Script	Command	Purpose
dev	<code>bun run dev</code>	Starts Vite dev server with hot reloading
build	<code>bun run build</code>	Compiles code and generates production bundles
preview	<code>bun run preview</code>	Spins up a local server to test the production build
lint	<code>bun run lint</code>	Analyzes code for linting errors using ESLint
format	<code>bun run format</code>	Standardizes styling layouts with Prettier

Shared Elder State (Mock Integration)

To simulate database interactions on the client side, the elder's custom profile details are persisted inside the browser's `localStorage` via handlers inside `elder-profile.ts`. Clear the browser's site data or use the log out action to reset mock details.

Deep Dive Documentation

- **🗺️ User Journey Guide:** Explore interactive flows, elder setup, and test the voice assistant step-by-step.
- **🔧 Technical Architecture & Stack:** Dive into API data handling, response routing, and the Text-to-Speech logic diagram.